Инструкция для системного администратора автоматизированной системы генерации персонализированных рецептов спортивного питания (АС ГРСН)

(составил студент Привалихин Дмитрий, ИСИб-23-1)

Содержание

1 Общие сведения 4
1.1 Назначение инструкции 4
1.2 Состав системы 4
1.3 Требования к серверу 5
2 Установка системы 6
2.1 Подготовка сервера 6
2.2 Получение дистрибутива 6
2.3 Автоматическая установка 6
2.4 Ручная установка 8
3 Настройка системы 8
3.1 Файл настроек 8
3.2 Подключение GigaChat 10
3.3 Настройка HTTPS 10
3.4 Резервные копии на отдельном диске 10
4 Сетевые доступы 11
4.1 Схема подключения и границы доступа 11
4.2 Настройка разрешений на подключение 12
4.3 Проверка сетевых доступов 12
5 Аутентификация и разграничение прав 12
5.1 Уровни доступа и ответственные лица 12
5.2 Вход в приложение и пользовательская сессия 13
5.3 Матрица прав в приложении 14
5.4 Выдача, изменение и прекращение доступа 14
5.5 Хранение и смена секретов 15
6 Доступы для подрядчиков 16
6.1 Основание и состав разрешения 16
6.2 Выбор минимального доступа 16
6.3 Выдача и контроль доступа 17
6.4 Отзыв доступа и закрытие работ 17
7 Доступ к другим системам и внешним сервисам 18
7.1 Реестр интеграций и зависимостей 18
7.2 Авторизация и проверка GigaChat 18
7.3 Получение кода и обновлений 19
8 Проверка работоспособности 19
9 Резервное копирование и восстановление 20
9.1 Автоматическое копирование 20
9.2 Создание копии вручную 20
9.3 Восстановление базы данных 21
9.4 Полнота копии и перенос на другой сервер 22
10 Работа с базой данных 23
10.1 Учётные записи и аутентификация PostgreSQL 23

10.2 Локальное подключение через клиент БД 24
10.3 Удалённое подключение через SSH-туннель 24
10.4 Персональная роль для чтения справочников 25
10.5 Отзыв прав и контроль изменений 25
11 Модуль администрирования 26
11.1 Управление учётными записями 27
11.2 Наполнение каталога и справочников 28
11.3 Особенность штатного начального наполнения 29
11.4 Просмотр журнала событий 29
11.5 Проверка целостности и исправление замечаний 30
12 Обслуживание системы 31
13 Типовые неисправности 32
14 Контроль готовности и журнал доступов 33
14.1 Контроль перед передачей в эксплуатацию 33
14.2 Учёт выдачи и отзыва прав 34
15 Источники технических сведений 35

1 Общие сведения

1.1 Назначение инструкции

Инструкция предназначена для системного администратора организации, которая приобрела автоматизированную систему генерации персонализированных рецептов спортивного питания (АС ГРСН). В инструкции описаны подготовка сервера, установка и первый запуск системы, настройка параметров, проверка работоспособности, резервное копирование и восстановление базы данных, обслуживание и устранение типовых неисправностей.

Инструкция охватывает доступ к веб-приложению, серверу, базе данных и внешним сервисам, а также выдачу и отзыв прав сотрудников и подрядчиков. Работа с пользователями, справочниками и журналом событий выполняется в модуле администрирования (раздел 11). Работа обычных пользователей описана в отдельной инструкции.

В документе различаются реализованные возможности приложения и организационный порядок эксплуатации. VPN, SSH-сервер, персональные учётные записи операционной системы и средства хранения секретов настраивает организация: они не входят в состав контейнеров АС ГРСН. Примеры адресов и имён необходимо заменить значениями своего сервера; действительные пароли и ключи в инструкцию не включаются.

1.2 Состав системы

Система поставляется в виде исходного кода с файлами сборки и разворачивается в контейнерах Docker. Состав контейнеров приведён в таблице 1, схема взаимодействия – на рисунке 1.

Таблица 1 – Состав контейнеров АС ГРСН

Контейнер	**Образ, технологии**	**Назначение**
db	PostgreSQL 15 (Alpine Linux)	База данных системы. Данные хранятся в томе Docker pgdata
backend	Python 3.12, FastAPI, Uvicorn (Debian 12)	REST API: бизнес-логика, расчёты, обращение к модели GigaChat, формирование отчётов, копирование и восстановление базы. При запуске применяет миграции и заполняет справочники
frontend	Nginx 1.27 (Alpine Linux)	Веб-интерфейс и обратный прокси: отдаёт страницы и передаёт запросы /api в backend; работает по HTTP или HTTPS
backup	PostgreSQL 15 (Alpine Linux), cron	Ежедневное резервное копирование базы утилитой pg_dump и удаление старых копий

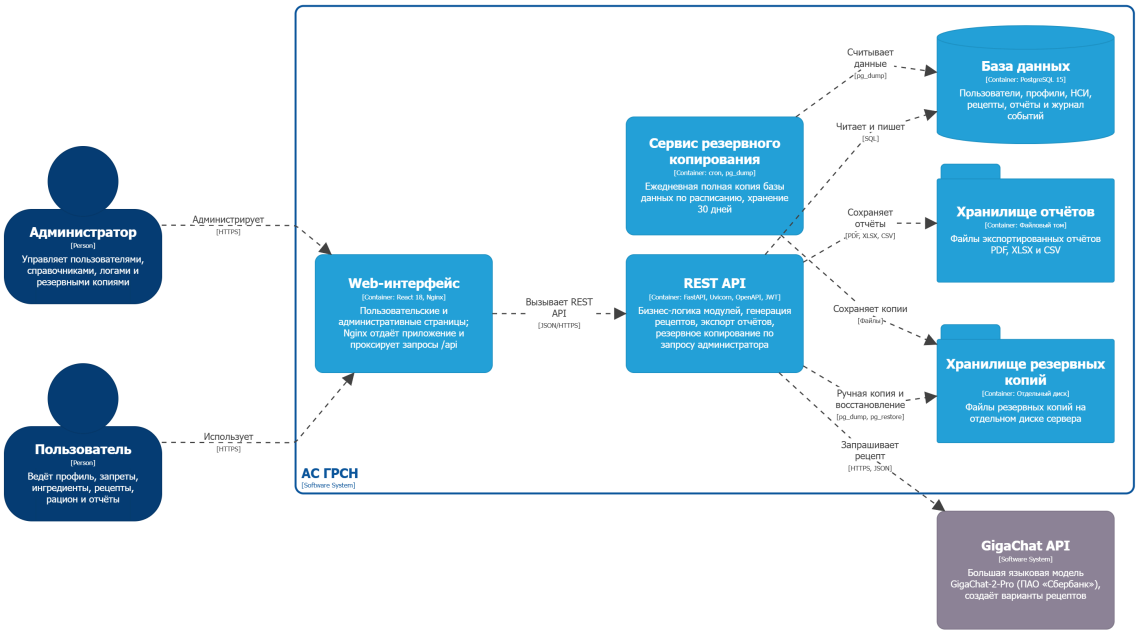


Рисунок 1 – Схема контейнеров АС ГРСН

Внешняя система – GigaChat API (ПАО «Сбербанк»): модель GigaChat-2-Pro формирует варианты рецептов. Если ключ не задан или сервис недоступен, при включённом резервном режиме используется встроенный локальный генератор (раздел 7.2).

1.3 Требования к серверу

Требования к серверу соответствуют техническому заданию:

– процессор – от 2 ядер;
– оперативная память – от 4 ГБ;
– дисковое пространство – от 20 ГБ на системном диске;
– отдельный диск (или отдельный раздел другого физического диска) для резервных копий;
– операционная система, на которой работает Docker: Linux-сервер с Docker Engine и плагином Docker Compose либо Windows 10/11 или macOS с Docker Desktop;
– доступ в интернет для загрузки образов при установке и для обращения к GigaChat API (адреса `ngw.devices.sberbank.ru`, порт 9443, и `gigachat.devices.sberbank.ru`, порт 443);
– свободные сетевые порты: для веб-интерфейса (по умолчанию 8080 – HTTP и 8443 – HTTPS) и порт базы данных для локального подключения администратора (по умолчанию 5432, доступен только с самого сервера).

Лимиты контейнеров приведены в таблице 2. Их сумма равна 2 ядрам и 4 ГБ; операционной системе и Docker также требуются ресурсы, поэтому для эксплуатации следует предусмотреть запас. Само наличие лимитов не подтверждает прохождение нагрузочных испытаний. Значения задаются в `.env` (раздел 3).

Таблица 2 – Ресурсы контейнеров

Контейнер	**Процессор, ядер**	**Память**	**Параметры в .env**
db	0,75	1,5 ГБ	DB_CPUS, DB_MEMORY
backend	1	1,75 ГБ	BACKEND_CPUS, BACKEND_MEMORY
frontend	0,15	256 МБ	FRONTEND_CPUS, FRONTEND_MEMORY
backup	0,1	512 МБ	BACKUP_CPUS, BACKUP_MEMORY
Итого	**2**	**4 ГБ**	

2 Установка системы

2.1 Подготовка сервера

1) Установите Docker:
   - на Linux-сервере – Docker Engine и плагин Docker Compose по официальной инструкции https://docs.docker.com/engine/install/;
   - на Windows или macOS – Docker Desktop (https://www.docker.com/products/docker-desktop/). После установки запустите Docker Desktop и дождитесь состояния «Engine running».
2) Установите Git (https://git-scm.com/).
3) Подключите и подготовьте отдельный диск для резервных копий, создайте на нём папку, например `/mnt/backup/grsn` на Linux или `D:\grsn_backups` на Windows. На Linux выдайте права на запись пользователю с идентификатором 1001, под которым работает API:

```
sudo mkdir -p /mnt/backup/grsn
sudo chown 1001:1001 /mnt/backup/grsn
```

4) Проверьте, что Docker работает:

```
docker info
docker compose version
```

2.2 Получение дистрибутива

Загрузите дистрибутив системы и перейдите в его каталог:

```
git clone --branch 2.0mb_norm https://github.com/Skipl1/ASOIU.git
cd ASOIU/Optimization-Methods/grsn_app
```

Все дальнейшие команды `docker compose` выполняются в этом каталоге (`grsn_app`).

2.3 Автоматическая установка

Для первоначального ознакомления запустите скрипт установки. На рабочем сервере сначала подготовьте `.env` по разделу 2.4: скрипт начинает запуск сразу и сам не заменяет стандартный пароль администратора, не отключает демонстрационного пользователя и не выбирает отдельный диск для копий.

```
# Linux, macOS, Git Bash
bash scripts/setup.sh

# Windows (PowerShell)
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
```

Скрипт выполняет следующие действия:

1) проверяет, что Docker установлен и запущен (на Windows при необходимости запускает Docker Desktop);
2) создаёт файл настроек `.env` из шаблона `.env.example`: генерирует случайные пароль базы данных, пароль учётной записи приложения и секрет подписи токенов (`JWT_SECRET`), подбирает свободные порты;
3) собирает и запускает контейнеры (`docker compose up -d --build`); первая сборка занимает 5–15 минут;
4) ждёт готовности системы (до 5 минут) и проверяет вход;
5) выводит адрес сайта, учётные записи и основные команды управления.

Параметры скрипта приведены в таблице 3.

Таблица 3 – Параметры скрипта установки

Linux / macOS	**Windows**	**Назначение**
--gigachat-key "<ключ>"	-GigaChatKey "<ключ>"	Записать ключ GigaChat в .env
--https	-Https	Включить HTTPS
--no-browser	-NoBrowser	Не открывать браузер после установки
--reset	-Reset	Удалить контейнеры и именованные тома с данными. Копии в подключённой папке хоста остаются; .env сохраняется

При повторном запуске скрипт сохраняет существующие секреты и порты; переданный ключ GigaChat и флаг HTTPS обновляют соответствующие настройки. Пользовательская база не очищается без параметра сброса. Однако при каждом старте `backend` выполняется начальное наполнение: штатные записи справочников с известными кодами записываются заново (раздел 11.3). Перед обновлением или повторной установкой создайте копию базы.

При установке сразу с HTTPS проверки готовности в поставляемых скриптах могут завершиться ошибкой: они обращаются к HTTP, который перенаправляется на HTTPS, а самоподписанный сертификат не доверен. В таком случае выполните ручную проверку по разделу 8. Для рабочего сервера используйте ручную установку с заранее подготовленным доверенным сертификатом.

Важно. Учётная запись администратора системы создаётся один раз – при первом запуске – из параметров `ADMIN_EMAIL`, `ADMIN_PASSWORD` и `ADMIN_NAME`. Перед первым запуском в рабочей среде задайте в `.env` собственный адрес и надёжный пароль администратора и отключите демонстрационные данные (`SEED_DEMO=false`), см. раздел 3. При каждом запуске контейнер `backend` проверяет, есть ли пользователь с адресом `ADMIN_EMAIL`, и создаёт его, только если такого нет. Поэтому изменение `ADMIN_PASSWORD` после первого запуска пароль

существующего администратора не меняет, а указание нового адреса ADMIN_EMAIL приводит к созданию ещё одной учётной записи администратора.

2.4 Ручная установка

Для управляемой установки выполните следующие действия.

1) Создайте файл настроек из шаблона:

```
# Linux / macOS
cp .env.example .env

# Windows PowerShell
Copy-Item .env.example .env
```

2) Откройте .env в текстовом редакторе. Задайте собственные POSTGRES_PASSWORD, APP_DB_PASSWORD, JWT_SECRET, ADMIN_EMAIL, ADMIN_PASSWORD, ADMIN_NAME и BACKUP_PATH; отключите демо-данные: SEED_DEMO=false. Секреты БД выбирайте длинными случайными строками из латинских букв и цифр: они подставляются в URL подключения. Для рабочего сервера включите HTTPS и подготовьте сертификат до запуска (раздел 3.3). Пример генерации случайного секрета при наличии Python:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

3) Соберите и запустите контейнеры:

```
docker compose up -d --build
```

4) Откройте в браузере адрес http://<адрес сервера>:8080 (или https://<адрес сервера>:8443 при включённом HTTPS) и войдите под учётной записью администратора.

При запуске backend применяет миграции, создаёт ограниченную роль приложения и наполняет справочники. Проверьте данные первого входа по своему .env: скрипт установки выводит стандартные логин и пароль даже при их изменении. Если демонстрационная запись уже была создана, SEED_DEMO=false не удалит её; заблокируйте её через админку.

3 Настройка системы

3.1 Файл настроек

Все параметры системы задаются в файле .env в каталоге grsn_app. Файл содержит пароли и ключи, поэтому его нельзя передавать посторонним и добавлять в систему контроля версий. Основные параметры приведены в таблице 4.

Таблица 4 – Основные параметры файла .env

Параметр	**Назначение**
База данных	
POSTGRES_USER, POSTGRES_DB	Владелец и имя базы данных PostgreSQL
POSTGRES_PASSWORD	Пароль владельца базы данных (обязательный)

Параметр	Назначение
APP_DB_PASSWORD	Пароль ограниченной учётной записи приложения grsn_app; если не задан, вычисляется из POSTGRES_PASSWORD
Безопасность	
JWT_SECRET	Секрет подписи токенов доступа – длинная случайная строка
ACCESS_TOKEN_MINUTES	Срок действия JWT, минут (30); веб-клиент продлевает его при активности, см. раздел 5.2
MAX_FAILED_LOGINS, LOCKOUT_MINUTES	Число неудачных попыток входа и время блокировки, минут (5 и 15)
Модель генерации	
LLM_PROVIDER	gigachat – GigaChat API; mock – только встроенный генератор
GIGACHAT_CREDENTIALS	Ключ авторизации GigaChat API
GIGACHAT_MODEL	Модель: GigaChat-2-Pro (основная) или GigaChat-2
LLM_MAX_CONCURRENCY	Число одновременных запросов к модели (тариф Freemium – 1)
LLM_FALLBACK_TO_MOCK	Использовать резервный генератор при недоступности модели (true)
Начальное наполнение	
ADMIN_EMAIL, ADMIN_PASSWORD, ADMIN_NAME	Учётная запись администратора, создаваемая при первом запуске
SEED_DEMO	Создавать демонстрационного пользователя (true/false); в рабочей среде – false
Веб-сервер и сеть	
ENABLE_HTTPS	true – работа по HTTPS, HTTP перенаправляется на HTTPS
HTTP_PORT, HTTPS_PORT	Порты веб-интерфейса на сервере (8080 и 8443)
SERVER_NAME	Имя сервера (домен) для сертификата
DB_PORT	Порт PostgreSQL для локального подключения администратора (5432)
Резервное копирование и прочее	
BACKUP_PATH	Папка резервных копий на отдельном диске
BACKUP_CRON	Расписание копирования в формате cron (по умолчанию 0 3 * * * – ежедневно в 03:00)
BACKUP_RETENTION_DAYS	Срок хранения копий, дней (30)
APP_TIMEZONE	Часовой пояс (Europe/Moscow)
API_WORKERS	Число процессов API (2)
*_CPUS, *_MEMORY	Ресурсы контейнеров (таблица 2)

После изменения `.env` примените настройки командой:

```
docker compose up -d
```

Docker пересоздаст контейнеры с изменёнными параметрами. Том базы при этом сохраняется, но начальное наполнение справочников выполняется при каждом старте API. Изменение `POSTGRES_PASSWORD` в `.env` само по себе не меняет пароль уже созданной роли PostgreSQL. Порядок работы с секретами приведён в разделе 5.5.

3.2 Подключение GigaChat

1) Зарегистрируйтесь на портале https://developers.sber.ru и создайте проект GigaChat API.
2) В настройках проекта получите ключ авторизации (Authorization key).
3) Запишите ключ в `.env`: `GIGACHAT_CREDENTIALS=<ключ>`. Для тарифа для юридических лиц укажите соответствующее значение `GIGACHAT_SCOPE` и при необходимости увеличьте `LLM_MAX_CONCURRENCY`.
4) Перезапустите API:

```
docker compose up -d backend
```

Корневой сертификат для соединения с GigaChat входит в поставку. Поле `llmProvider` служебного адреса `/health` показывает выбранную настройку и не подтверждает успешную авторизацию во внешнем сервисе. Проверка фактической генерации описана в разделе 7.2.

3.3 Настройка HTTPS

1) Задайте в `.env` параметры `ENABLE_HTTPS=true` и `SERVER_NAME=<домен сервера>`.
2) Поместите сертификат и закрытый ключ, выданные для домена, в каталог `frontend/nginx/certs` под именами `server.crt` и `server.key`.
3) Перезапустите веб-сервер:

```
docker compose up -d frontend
```

Если файлы не найдены, создаётся самоподписанный сертификат. Он предназначен для учебной проверки; для рабочего сервера установите сертификат доверенного центра с корректным доменным именем. После включения HTTPS сайт доступен по адресу `https://<домен>:8443`, HTTP перенаправляется на HTTPS. Для стандартных портов задайте `HTTP_PORT=80`, `HTTPS_PORT=443` и примените настройки.

При замене только файлов сертификата выполните `docker compose restart frontend`: команда `up -d` может не перезапустить уже работающий Nginx. Контролируйте срок сертификата и сохраняйте закрытый ключ в защищённом каталоге. Параметр `SERVER_NAME` не создаёт запись DNS: имя сервера должно разрешаться средствами сети организации.

3.4 Резервные копии на отдельном диске

Задайте в `.env` путь к папке на отдельном диске, подготовленной в разделе 2.1, например `BACKUP_PATH=/mnt/backup/grsn` (Linux) или `BACKUP_PATH=D:/grsn_backups` (Windows), и выполните `docker compose up -d`. Если параметр не задан, копии хранятся в томе Docker на том же диске, что и база данных; такой режим не соответствует требованиям к надёжности.

4 Сетевые доступы

4.1 Схема подключения и границы доступа

Пользователь работает с АС ГРСН через браузер. Внешние запросы принимает `frontend` (Nginx): он выдаёт страницы и передаёт запросы `/api` в `backend`. Сервер приложения обращается к PostgreSQL и GigaChat. Браузеру пользователя прямой доступ к PostgreSQL и ключ GigaChat не требуются.

Docker Compose создаёт общую сеть проекта. В ней контейнеры обращаются друг к другу по именам сервисов: `backend:8000`, `db:5432`. Адрес `localhost` внутри контейнера указывает на сам контейнер. В поставке не настроены отдельные изолированные сети для каждой пары сервисов; перечисление соединений в таблице 5 означает необходимые потоки, а не наличие межконтейнерных правил фильтрации.

Таблица 5 – Необходимые сетевые соединения

Источник	**Назначение**	**Порт**	**Условия доступа**
Браузер пользователя	Сервер, Nginx	TCP 8443	HTTPS; разрешён пользователям системы. Порт задаётся HTTPS_PORT
Браузер пользователя	Сервер, Nginx	TCP 8080	HTTP: в рабочем режиме перенаправление на HTTPS; HTTP_PORT
frontend	backend	TCP 8000	REST API внутри сети Docker. Порт на хосте не публикуется
backend, backup	db	TCP 5432	Работа с БД и резервное копирование внутри Docker
Клиент БД на сервере	127.0.0.1 сервера	TCP 5432	Локальная публикация PostgreSQL; значение DB_PORT
Рабочее место администратора	Сервер ОС	TCP 22	Пример SSH для Linux, если он настроен организацией; только из согласованной сети
backend	ngw.devices. sberbank.ru	TCP 9443	Исходящее HTTPS-соединение для получения токена GigaChat
backend	gigachat.devices. sberbank.ru	TCP 443	Исходящее HTTPS-соединение для запросов к модели
Сервер / Docker	DNS организации	UDP/TCP 53	Разрешение имён сервисов интернета через настроенный резолвер

Порты 8080, 8443 и 5432 в таблице – значения по умолчанию. При установке скрипт может выбрать другие свободные порты. В правилах сети, ссылках для пользователей и клиентах БД используйте фактические значения `.env`. Внутренние порты контейнеров при этом остаются 80/443, 8000 и 5432.

4.2 Настройка разрешений на подключение

1) Определите доменное имя и адрес сервера, сети пользователей, рабочие места администраторов и адрес VPN-шлюза, если он используется.
2) Разрешите входящие соединения пользователей только на веб-порты. Для промышленной эксплуатации включите HTTPS; при стандартных портах это TCP 443 и, при необходимости перенаправления, TCP 80.
3) Доступ к SSH или другому средству управления сервером предоставьте только администраторам из согласованных адресов либо через VPN организации. Порт 22 в таблице – пример, а не установленная в АС ГРСН служба.
4) Сохраните публикацию БД на `127.0.0.1`. Не заменяйте её привязкой ко всем интерфейсам ради подключения подрядчика. Удалённое подключение выполняйте через защищённый туннель (раздел 10.3).
5) Разрешите исходящие соединения к GigaChat и DNS. Загрузку кода, контейнерных образов и зависимостей разрешайте на время установки и обновлений (раздел 7.3).
6) Проверьте правила на внешнем межсетевом экране и на хосте, включая IPv6. На Linux Docker создаёт собственные правила публикации портов; одной настройки UFW может оказаться недостаточно. Не отключайте управление сетью Docker без отдельного проекта сетевой конфигурации.

Публикация веб-портов в Compose не ограничена конкретным интерфейсом хоста. Поэтому ограничение по сетям пользователей и администраторов необходимо проверить фактическим подключением с разрешённого и запрещённого компьютера. Точная команда настройки межсетевого экрана зависит от ОС и сетевого оборудования организации.

4.3 Проверка сетевых доступов

На рабочем месте Windows проверьте веб-порт и закрытость прямого доступа к БД. В примере `grsn.example.org` – имя вашего сервера:

```
Test-NetConnection grsn.example.org -Port 8443
Test-NetConnection grsn.example.org -Port 5432
```

Для разрешённого рабочего места веб-порт должен быть доступен. Прямой порт БД с другого компьютера должен быть недоступен. Результат проверки TCP подтверждает только установление соединения; правильность сертификата, входа и прав проверяется отдельно.

На сервере проверьте `docker compose ps`: у БД должна быть привязка вида `127.0.0.1:5432->5432/tcp`, у `backend` не должно быть опубликованного порта на хосте. Затем откройте сайт с рабочего места и проверьте HTTPS. В журнале эксплуатации запишите проверенные адреса, порты, дату и результат.

5 Аутентификация и разграничение прав

5.1 Уровни доступа и ответственные лица

Аутентификация подтверждает личность пользователя, авторизация определяет допустимые действия. В АС ГРСН учётные записи приложения, операционной системы, PostgreSQL и внешних сервисов независимы.

Назначение роли администратора в веб-приложении не создаёт учётную запись ОС или PostgreSQL.

Руководитель организации или владелец системы согласовывает выдачу прав. Системный администратор настраивает сервер, сеть и БД, предоставляет доступ и фиксирует изменения. Администратор приложения ведёт пользователей и справочники через веб-интерфейс. Эти обязанности может выполнять один сотрудник, но при выдаче прав следует учитывать каждый уровень отдельно.

Таблица 6 – Уровни аутентификации и доступа

Уровень	**Способ проверки**	**Назначение и ограничение**
Веб-приложение	E-mail, пароль, затем JWT	Собственные данные пользователя; роль admin дополнительно открывает весь модуль администрирования
Сервер ОС	Персональная запись ОС; для SSH рекомендуется ключ	Установка, конфигурация и обслуживание. Настраивается организацией отдельно
PostgreSQL	Роль БД и пароль; правила pg_hba.conf	Права на БД и таблицы определяются в PostgreSQL, независимо от роли веб-пользователя
GigaChat	Ключ авторизации проекта, затем токен API	Межсервисное обращение от backend; пользователю браузера ключ не выдаётся
Репозиторий и поставка	Учётная запись сервиса, если доступ ограничен	Получение кода и обновлений; права записи выдаются только сопровождающим разработчикам

5.2 Вход в приложение и пользовательская сессия

1) Пользователь регистрирует персональную учётную запись с именем, уникальным e-mail и паролем. Регистрация создаёт обычного пользователя; предварительное одобрение регистрации в текущей версии не реализовано.
2) При входе сервер проверяет пароль и состояние учётной записи. Пароли хранятся как bcrypt-хеши. Минимум – 8 символов, обязательно наличие букв и цифр; предельная длина – 72 байта UTF-8.
3) После успешного входа сервер выдаёт JWT. В защищённых запросах клиент передаёт его в заголовке `Authorization: Bearer <token>`. Передача данных выполняется по HTTPS.
4) Срок JWT определяется `ACCESS_TOKEN_MINUTES` (по умолчанию 30 минут). Веб-клиент периодически продлевает токен, пока фиксирует активность пользователя. Непродлённый токен истекает; фактическое завершение сессии связано со сроком последнего выданного токена.
5) После `MAX_FAILED_LOGINS` неудачных попыток (по умолчанию 5) вход в данную учётную запись блокируется на `LOCKOUT_MINUTES` (15 минут). Это блокировка по учётной записи; отдельный лимит по IP в приложении не реализован.

6) Команда «Выйти» отзывает все ранее выданные токены этой учётной записи. Изменение роли или статуса администратором также меняет версию токена и завершает существующие сессии.

С флажком «Запомнить устройство» сессия сохраняется в локальном хранилище браузера; без него – в хранилище вкладки/сеанса. Флажок не отменяет срок JWT. На общедоступных компьютерах завершайте работу кнопкой «Выйти».

Встроенная аутентификация не использует LDAP, Active Directory, корпоративный SSO или второй фактор. Если организация требует такие механизмы, их внедрение требует отдельной настройки инфраструктуры и согласованной доработки; наличие HTTPS само по себе их не обеспечивает.

5.3 Матрица прав в приложении

Таблица 7 – Матрица прав пользователей АС ГРСН

Операция	**Пользователь**	**Администратор**
Ведение профиля, запретов и набора продуктов	Свои данные	Свои данные
Генерация, история, рацион, отчёты, анализ	Свои данные	Свои данные
Список пользователей, их роли, блокировка и удаление	Нет	Да
Изменение ингредиентов, категорий и аллергенов	Нет	Да
Журнал событий, активность, проверка целостности	Нет	Да
Создание копии и восстановление всей БД	Нет	Да
Управление ОС, прямой SQL, ключи внешних сервисов	По отдельному разрешению	По отдельному разрешению

Роль `admin` предоставляет все административные функции сразу. Отдельных ролей «только просмотр логов», «редактор каталога» и «подрядчик» нет. Проверку доступа выполняет сервер для каждого защищённого запроса; скрытие пункта меню не является единственной мерой разграничения.

5.4 Выдача, изменение и прекращение доступа

1) Получите согласование владельца системы: сотрудник, задача, необходимая роль, дата начала и срок доступа.
2) Для работы в приложении используйте персональную регистрацию. Проверьте e-mail сотрудника и при необходимости назначьте роль через «Администрирование» → «Пользователи» → «Сделать админом».
3) Запишите в журнал выдачи доступов, кто согласовал права и кто их назначил. Событие изменения роли дополнительно появится в журнале приложения.
4) После изменения обязанностей снимите лишние права кнопкой «Снять админа». Если доступ больше не нужен, заблокируйте запись кнопкой «Блокировать».
5) Проверьте прекращение доступа: старый токен не должен выполнять защищённые запросы, вход заблокированной записи должен отклоняться.

Удаление учётной записи удаляет связанные пользовательские данные, поэтому для прекращения доступа обычно достаточно блокировки. Нельзя заблокировать, удалить или понизить собственную учётную запись через админку. Поддерживайте хотя бы одну действующую персональную запись ответственного администратора.

В интерфейсе нет функции самостоятельной смены или восстановления забытого пароля. Если доступ администратора утрачен, системный администратор может задать в `.env` новый, ранее не использовавшийся `ADMIN_EMAIL` и надёжный `ADMIN_PASSWORD`, затем выполнить `docker compose up -d backend`. После входа новой записью следует заблокировать старую и обновить реестр доступов. Изменение только `ADMIN_PASSWORD` не меняет пароль уже существующего пользователя.

5.5 Хранение и смена секретов

К секретам относятся пароли пользователей и БД, `JWT_SECRET`, ключ GigaChat, закрытый ключ HTTPS и ключи удалённого доступа. Храните их в принятом организацией защищённом хранилище; доступ к `.env`, каталогу сертификатов и резервным копиям предоставляйте только ответственным за сопровождение. В заявках, отчётах и снимках экрана используйте скрытые значения.

На Linux владелец каталога развёртывания может ограничить чтение `.env` командой `chmod 600 .env`; на Windows настройте права NTFS для ответственных учётных записей. Доступ к Docker и запуск команд с повышенными правами также предоставляют широкие возможности управления хостом и контейнерами; обычному пользователю приложения они не нужны.

Порядок смены секретов:

- `JWT_SECRET`: задайте новую случайную строку и пересоздайте `backend` командой `docker compose up -d backend`. Существующие JWT перестанут проходить проверку; пользователям потребуется войти снова.
- `APP_DB_PASSWORD`: измените значение в `.env` и пересоздайте `backend`. При старте служебная процедура обновит пароль роли `grsn_app` с помощью владельца БД. Выполняйте это в согласованное окно обслуживания.
- `POSTGRES_PASSWORD`: сначала смените фактический пароль владельца через защищённое соединение PostgreSQL, например командой `\password grsn` в `psql`, затем немедленно обновите `.env` и пересоздайте сервисы. Имя `grsn` замените своим `POSTGRES_USER`. До начала работ подготовьте доступ администратора к серверу и копию БД: одна правка `.env` оставит работающую базу со старым паролем.
- Ключ GigaChat: перевыпустите ключ в проекте внешнего сервиса, запишите новое значение в `.env`, пересоздайте `backend` и проверьте фактическую генерацию.
- Закрытый ключ HTTPS: замените файлы сертификата и ключа и перезапустите `frontend`; при компрометации отзовите прежний сертификат у выдавшего его центра.

Если `APP_DB_PASSWORD` не задан явно, он выводится из `POSTGRES_PASSWORD`; учитывайте эту зависимость при смене пароля владельца. Пароли прикладных пользователей, владельца БД и сервисных ролей не должны совпадать.

6 Доступы для подрядчиков

6.1 Основание и состав разрешения

Подрядчик – внешний специалист, выполняющий согласованные работы по сопровождению. В текущей версии приложения нет отдельного типа учётной записи подрядчика и автоматического срока действия веб-доступа. Следующий порядок является регламентом организации; его выполняет ответственный администратор.

До начала работ оформите заявку. Обязательные сведения приведены в таблице 8. Секреты в заявку не включаются.

Таблица 8 – Содержание заявки на доступ подрядчика

Сведение	**Что указать**
Исполнитель	Организация, Ф. И. О., персональный e-mail, контакт для связи
Основание	Номер договора/заявки, конкретная задача и ожидаемый результат
Ресурсы и права	Тестовый или рабочий контур; приложение, сервер, БД, репозиторий; требуемые операции
Сетевой доступ	Разрешённый адрес/подсеть, VPN или иной согласованный способ подключения
Срок	Дата и время начала и окончания с часовым поясом; окно обслуживания
Ответственные	Кто согласовал доступ, кто его выдал, кто сопровождает работы и кто отзывает права

6.2 Выбор минимального доступа

– Проверка интерфейса: отдельная запись обычного пользователя и тестовые данные. Доступ к ОС, БД и ключам не требуется.
– Диагностика ошибок: ответственный сотрудник предоставляет необходимые журналы с удалёнными секретами; прямой доступ открывается только при обоснованной необходимости.
– Работа со справочниками: предпочтителен тестовый контур. Если требуется роль `admin` в рабочем приложении, согласуйте её полный объём: эта роль также позволяет управлять пользователями и восстанавливать БД.
– Диагностика БД: персональная роль с правами чтения только нужных справочных таблиц, сроком пароля и защищённым подключением (раздел 10.4).
– Обновление приложения: персональный доступ к серверу на время работ либо выполнение команд штатным администратором под наблюдением подрядчика. Изменение кода согласуется отдельно от доступа к рабочей базе.

Сервисные роли `grsn_app` и владелец БД не используются как общие учётные записи подрядчиков. Для воспроизведения ошибки сначала подготовьте копию тестовой среды с обезличенными данными; дамп рабочей БД содержит хеши паролей, профили, ограничения и историю пользователей.

6.3 Выдача и контроль доступа

1) Проверьте заявку и согласование владельца системы. Создайте персональные записи только в тех системах, которые нужны для задачи.
2) Для удалённой работы выдайте доступ через инфраструктуру организации. Если используется SSH, установите публичный ключ исполнителя в разрешённую запись ОС; закрытый ключ остаётся у исполнителя. Для подключения только к БД ограничьте назначение туннеля, если это поддерживает выбранная конфигурация SSH.
3) Установите срок доступа в журнале и напоминание ответственному. Веб-приложение само не заблокирует запись по дате окончания заявки.
4) Передайте параметры подключения через согласованный канал. Пароль или ключ передавайте отдельно от описания адреса и учётной записи.
5) Совместно проверьте разрешённые операции и отсутствие лишних прав. Перед изменениями рабочей среды создайте резервную копию и согласуйте окно работ.
6) Фиксируйте время работ и изменения. Действия в админке проверяйте во вкладке «Логи»; входы в ОС – в журналах ОС. Прямые SQL-запросы не попадают в `audit_log` приложения: результаты и команды фиксируются отдельно в отчёте работ.

В АС ГРСН нет встроенной системы управления заявками, записи SSH-сеансов или полного аудита SQL. Организация выбирает эти средства отдельно; журнал приложения покрывает только события, которые записывает само приложение.

6.4 Отзыв доступа и закрытие работ

1) Получите отчёт исполнителя, проверьте результат и сохранность данных.
2) Снимите роль администратора и заблокируйте веб-запись подрядчика. Это отзовёт её действующие JWT; изменение одного пароля ОС не прекращает веб-сессию.
3) Заблокируйте запись PostgreSQL и завершите её активные соединения по разделу 10.5. Истечение `VALID UNTIL` не закрывает уже открытые подключения.
4) Удалите разрешение VPN, публичный SSH-ключ, временные правила сети и права репозитория; завершите открытые сеансы средствами соответствующей системы.
5) Если исполнителю передавались сервисные секреты, смените их по разделу 5.5. Получите подтверждение удаления временных копий данных и конфигурации с его рабочего места.
6) Повторите проверки подключения: доступ должен быть закрыт на каждом выданном уровне. Запишите дату, исполнителя отзыва и результат в заявку/журнал.

При подозрении на компрометацию доступ отзывается немедленно, без ожидания окончания срока. Ответственный сохраняет журналы, сообщает владельцу системы и проверяет изменения данных.

7 Доступ к другим системам и внешним сервисам

7.1 Реестр интеграций и зависимостей

Основная внешняя интеграция приложения – GigaChat API. Остальные перечисленные системы обслуживают поставку и инфраструктуру. Доступ к ним выдаётся отдельно от роли в АС ГРСН.

Таблица 9 – Другие системы, используемые при эксплуатации

Система	Назначение и доступ	Ответственный и контроль
GigaChat API	Генерация рецептов; ключ проекта и токен, исходящие TCP 9443/443	Ответственный за интеграцию: ключ, доступные модели, квоты и ошибки
Репозиторий GitHub	Получение кода по HTTPS; для закрытого репозитория – персональные права чтения	Сопровождающий: разрешённая версия, права записи, отзыв доступа подрядчика
Реестры образов и пакетов	Docker Hub, PyPI, npm, репозитории ОС при сборке; исходящий HTTPS	Системный администратор: доступность загрузок, согласованные версии и источник поставки
DNS, время, сертификаты	Службы организации для имён, точного времени и HTTPS	Системный администратор: корректность имени, времени хоста и срока сертификата
VPN / SSH	Удалённое сопровождение, если развёрнуто организацией	Администратор инфраструктуры: персональные записи, адреса, журналы и отзыв

Учётные записи почтового сервера, LDAP/AD и корпоративного SSO приложением не используются. E-mail в АС ГРСН служит логином: письмо подтверждения и ссылка восстановления пароля не отправляются. pgAdmin и DBeaver – необязательные клиентские инструменты администратора, а не дополнительные серверные компоненты АС ГРСН.

7.2 Авторизация и проверка GigaChat

Ключ `GIGACHAT_CREDENTIALS` принадлежит проекту внешнего сервиса. SDK получает по нему токен доступа и использует его в запросах к API. Учётные данные пользователя АС ГРСН не являются учётными данными GigaChat. Параметр `GIGACHAT_SCOPE` должен соответствовать типу проекта поставщика.

Во внешний запрос передаются цель, целевые калории и БЖУ, доступные продукты, ограничения, пожелания и названия ранее оценённых рецептов. Код формирования запроса не добавляет e-mail, пароль или JWT пользователя; однако введённые пользователем пожелания и ограничения также требуют согласованного порядка обработки данных. Не используйте реальные пользовательские сведения для проверки интеграции без разрешённого основания.

Проверка после настройки ключа:

1) Разрешите сетевые соединения из раздела 4.1 и примените настройки `backend`.
2) Войдите тестовым пользователем, заполните профиль и набор как минимум из трёх допустимых ингредиентов, включая источник белка.

3) Выполните генерацию. Проверьте сведения о модели в результате: должен использоваться `gigachat`, а не резервный `mock`; предупреждение о локальном генераторе означает, что внешняя генерация не подтверждена.
4) При ошибке изучите `docker compose logs --tail 100 backend`. Проверьте ключ, соответствие scope, доступность сети, доверие сертификату, доступность модели и квоту проекта.
5) Зафиксируйте результат и дату проверки без сохранения ключа в отчёте.

Ответ `/health` с `status=ok` подтверждает работу API и запрос к БД; поля `llmProvider` и `llmModel` возвращают настройки. Успешный ответ не доказывает доступность GigaChat, правильность ключа и наличие квоты. Для рабочего соединения оставляйте проверку TLS включённой (`GIGACHAT_VERIFY_SSL_CERTS=true`).

При `LLM_FALLBACK_TO_MOCK=true` сбой внешнего сервиса приводит к использованию локального генератора с предупреждением пользователю, если удаётся сформировать допустимый рецепт. Если резерв отключён, запрос может завершиться отказом. `LLM_MAX_CONCURRENCY`, время ожидания очереди и доступные модели должны соответствовать условиям фактического проекта GigaChat; тарифные условия проверяйте у поставщика.

7.3 Получение кода и обновлений

Для клонирования используется HTTPS-доступ к GitHub. Если репозиторий закрыт, выдайте сопровождающему персональное право чтения; возможность менять код предоставляется отдельно. Во время сборки Docker загружает базовые образы, зависимости Python и JavaScript, пакеты ОС. Помимо основного адреса реестра могут требоваться его адреса авторизации и доставки файлов; точный список согласуйте по выбранным источникам и журналу сборки.

При отсутствии прямого выхода в интернет используйте согласованный прокси или внутренние зеркала и заранее подготовленные образы. Эти средства в поставке АС ГРСН не настроены. Сетевые правила для обновления не должны автоматически давать внешним исполнителям доступ к рабочей БД или секретам.

Для каждого внешнего сервиса в эксплуатационном журнале укажите владельца учётной записи, назначение доступа, место хранения секрета, ответственного за смену и порядок отзыва. После смены подрядчика проверьте его права во внешних системах независимо от блокировки записи в АС ГРСН.

8 Проверка работоспособности

1) Проверьте состояние контейнеров:

```
docker compose ps
```

Все четыре контейнера должны быть в состоянии `Up`, контейнеры `db` и `backend` – с отметкой `(healthy)`:

```
NAME              SERVICE    STATUS
grsn-backend-1    backend    Up (healthy)
grsn-backup-1     backup     Up
grsn-db-1         db         Up (healthy)
grsn-frontend-1   frontend   Up
```

2) Проверьте служебный адрес проверки доступности API и базы данных:

```
# HTTP, Linux / macOS
curl http://localhost:8080/api/v1/health
# HTTP, Windows PowerShell
curl.exe http://localhost:8080/api/v1/health
```

Ожидается JSON с `status: "ok"`. Для HTTPS используйте действительный адрес сайта, например `https://grsn.example.org:8443/api/v1/health`, и доверенный сертификат. Не отключайте проверку сертификата при рабочей эксплуатации. Значения `llmProvider` и `llmModel` показывают настройки, а не результат проверки внешнего сервиса.

3) Проверьте ресурсы контейнеров:

```
bash scripts/show_limits.sh
# Windows:
powershell -ExecutionPolicy Bypass -File scripts\show_limits.ps1
```

Скрипт выводит лимиты процессора и памяти каждого контейнера, текущее потребление и итог (2 ядра, 4 ГБ). Нагрузку в реальном времени показывает команда `docker stats`.

4) Откройте сайт в браузере, войдите под учётной записью администратора и убедитесь, что открывается раздел «Администрирование» (раздел 11). Документация API доступна по адресу `/api/docs`.

5) Проверьте разграничение прав: без входа защищённый запрос должен возвращать HTTP 401, а с JWT обычного пользователя запрос `/api/v1/admin/users` – HTTP 403. Убедитесь, что обычный пользователь не получает чужие рецепты по прямому адресу API. Реальные токены в журнал проверки не записывайте.

6) Проверьте сетевые разрешения по разделу 4.3 и внешнюю генерацию по разделу 7.2. Создайте резервную копию и проверьте наличие файла и успешное событие в журнале; сообщение о запуске задания ещё не означает завершения копирования.

Служебный адрес `/health` не требует входа. Доступ к нему и к Swagger UI определяется сетевым размещением сервера; отсутствие входа в браузере не означает, что все пути `/api` закрыты.

9 Резервное копирование и восстановление

9.1 Автоматическое копирование

Контейнер `backup` по расписанию `BACKUP_CRON` (по умолчанию ежедневно в 03:00) создаёт полную копию базы данных утилитой `pg_dump` в формате PostgreSQL custom. Файлы называются `grsn_<дата>_<время>_auto.dump` и сохраняются в папку `BACKUP_PATH`. Копии старше `BACKUP_RETENTION_DAYS` дней удаляются автоматически. Результат каждого запуска записывается в журнал контейнера:

```
docker compose logs backup
```

9.2 Создание копии вручную

Внеплановую копию можно создать двумя способами:

– в веб-интерфейсе: «Администрирование» → вкладка «Резервные копии» → кнопка «Создать копию» (рисунок 2). Копия получит имя `grsn_<дата>_<время>_manual.dump`, событие запишется в журнал событий системы;
– из консоли сервера:

```
docker compose exec backup sh /scripts/backup.sh
```

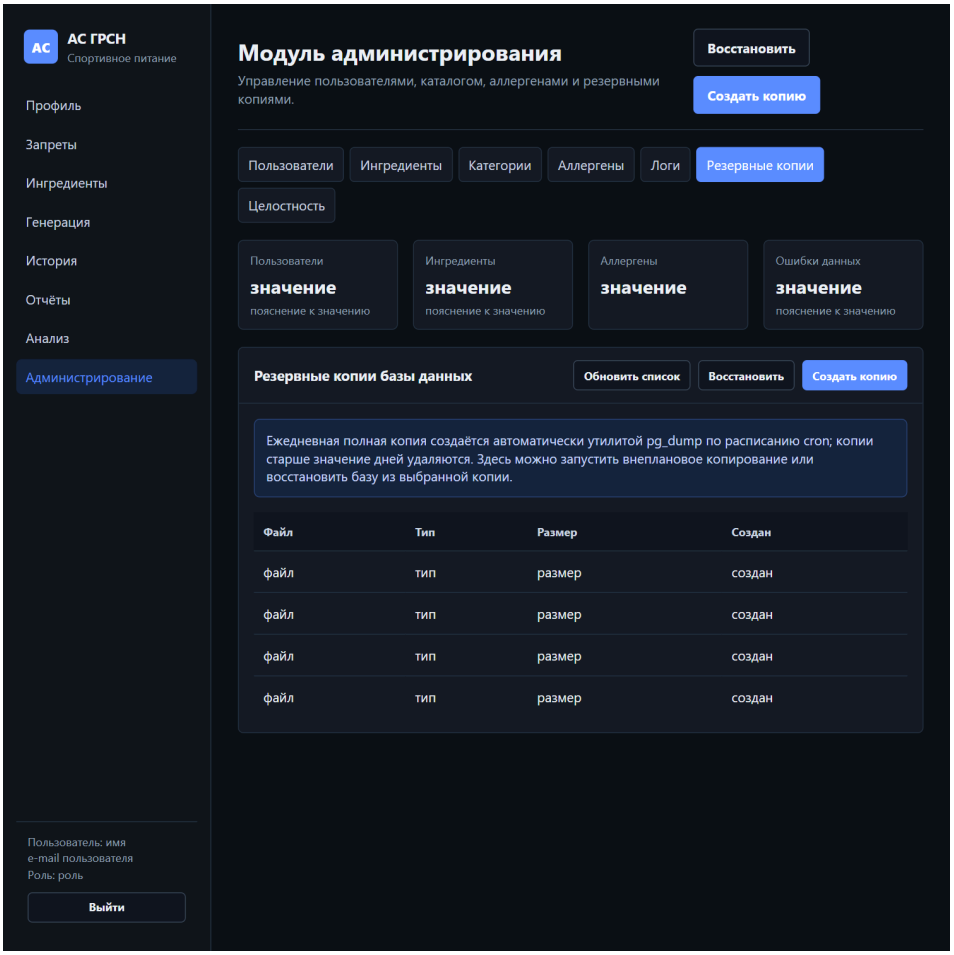


Рисунок 2 – Вкладка «Резервные копии»

Веб-интерфейс сначала сообщает «Резервное копирование запущено»: задача выполняется в фоне. Через некоторое время нажмите «Обновить список», проверьте ненулевой размер файла и событие `backup_create` с результатом `ok` во вкладке «Логи». При ошибке изучите журнал `backend`. Запуск `backup.sh` из консоли создаёт файл с суффиксом `_auto`, как при запуске по расписанию, и пишет результат в вывод скрипта; отдельную запись `audit_log` этот скрипт не создаёт.

9.3 Восстановление базы данных

Восстановление заменяет все данные системы состоянием на момент создания копии. Перед восстановлением рекомендуется создать свежую копию текущего состояния.

Восстановление из веб-интерфейса:

1) Откройте «Администрирование» и нажмите кнопку «Восстановить».
2) В окне «Восстановление из резервной копии» (рисунок 3) выберите копию в списке «Резервная копия».

3) Нажмите кнопку «Восстановить» и дождитесь сообщения о результате. Пользователям может потребоваться войти в систему заново.

Перед подтверждением ограничьте доступ пользователей и согласуйте окно обслуживания. В веб-интерфейсе после выбора копии появляется дополнительное подтверждение замены данных. Восстановление из веб-интерфейса не включает автоматический перевод системы в режим обслуживания.

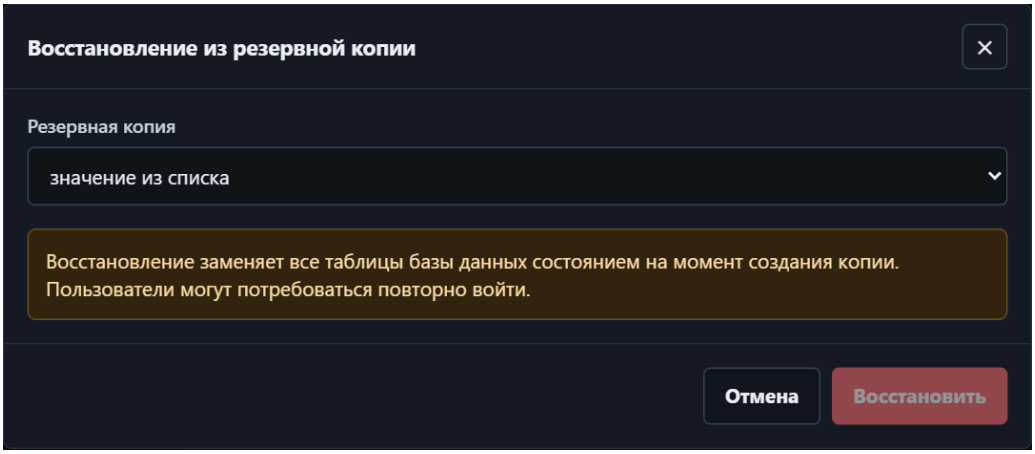


Рисунок 3 – Окно «Восстановление из резервной копии»

Для восстановления из консоли остановите обработку пользовательских запросов. Сначала выполните на сервере:

```
docker compose exec backup ls /backups
docker compose stop frontend backend
docker compose exec backup sh
```

В открывшейся оболочке контейнера `backup` выполните следующие команды. `BACKUP_FILE.dump` замените именем выбранной копии. Переносы с `\` относятся к оболочке контейнера и одинаково применимы при запуске из PowerShell или Linux:

```
pg_restore -h "$PGHOST" -U "$POSTGRES_USER" \
  -d "$POSTGRES_DB" --clean --if-exists --no-owner \
  --single-transaction /backups/BACKUP_FILE.dump
echo $?
exit
```

Код завершения `0` означает успешное выполнение утилиты. При ненулевом коде изучите ошибку; не считайте восстановление завершённым. После успешного восстановления запустите сервисы:

```
docker compose up -d backend frontend
```

Проверьте вход, профиль, историю, справочники и отчёты. При старте API начальное наполнение снова запишет штатные справочники (раздел 11.3), поэтому результат восстановления этих записей необходимо проверить отдельно. Прямой запуск `pg_restore` не создаёт событие восстановления в журнале приложения; зафиксируйте его в эксплуатационном журнале.

9.4 Полнота копии и перенос на другой сервер

Дамп содержит данные и структуру PostgreSQL. Он не включает файлы экспортированных отчётов в томе `storage`, `.env`, ключи и сертификаты HTTPS,

конфигурацию хоста и определения ролей PostgreSQL уровня кластера. Для полного восстановления сохраняйте эти компоненты отдельно с ограничением доступа. Отсутствие файла отчёта после восстановления БД может привести к статусу «файл удалён», хотя запись отчёта в БД сохранена.

Для переноса установите на новом сервере согласованную версию приложения, подготовьте роли и секреты, перенесите копию в `BACKUP_PATH`, восстановите БД и хранилище файлов, затем проверьте DNS, сертификат и сетевые разрешения. Роли, на которые ссылаются права в дампе, должны существовать на новом сервере; дополнительные персональные роли создаются администратором отдельно. Не используйте вместе новую несовместимую схему и старую версию приложения без проверки миграций.

Не реже одного раза в месяц и перед значимым обновлением выполняйте пробное восстановление в отдельной среде. Записывайте имя копии, время начала и окончания, результат проверки и исполнителя. Ежедневное копирование допускает потерю изменений после последней успешной копии; допустимый срок восстановления и частоту дополнительных копий согласует владелец системы.

10 Работа с базой данных

10.1 Учётные записи и аутентификация PostgreSQL

Приложение работает под сервисной ролью `grsn_app`. Ей разрешены `SELECT`, `INSERT`, `UPDATE`, `DELETE` в схеме `public`, кроме таблицы `alembic_version`; права изменения структуры таблиц не выдаются. Пароль берётся из `APP_DB_PASSWORD` либо выводится из `POSTGRES_PASSWORD`. Эта роль предназначена для API, а не для персональных подключений сотрудников.

Миграции, начальное наполнение и резервное копирование используют владельца из `POSTGRES_USER` (по умолчанию `grsn`). В поставляемом образе PostgreSQL эта начальная роль обладает полномочиями суперпользователя, поэтому её реквизиты доступны только ответственному за сопровождение. Право администратора веб-приложения не даёт права узнать пароль БД.

Контейнер `backend` получает и ограниченную строку `DATABASE_URL`, и привилегированную `OWNER_DATABASE_URL` для операций сопровождения. Ограничение роли `grsn_app` относится к обычным запросам приложения; доступ к окружению контейнера и командам Docker следует считать привилегированным.

Сетевые подключения БД проверяются средствами PostgreSQL по роли, паролю и правилам `pg_hba.conf`. Точный метод аутентификации действующего кластера проверяет администратор; приложение не управляет этим файлом. Подключение через локальный сокет внутри контейнера может использовать иной метод, чем TCP. Если пользователь может выполнять `docker compose exec db`, он уже имеет высокий уровень доступа к инфраструктуре.

Для проверки действующих правил в административной сессии `psql`:

```
SELECT type, database, user_name, address, auth_method
FROM pg_hba_file_rules
WHERE error IS NULL;
```

Не заменяйте парольную проверку сетевых подключений режимом `trust`. Внутреннее соединение PostgreSQL в поставке не настроено на TLS; для

удалённого администрирования защищайте участок между рабочим местом и сервером SSH-туннелем или VPN организации.

10.2 Локальное подключение через клиент БД

Установите pgAdmin, DBeaver или другой клиент PostgreSQL на компьютер администратора. Если клиент запущен на самом сервере, создайте подключение со следующими параметрами:

- хост – `127.0.0.1`;
- порт – значение `DB_PORT` (по умолчанию 5432);
- база – значение `POSTGRES_DB` (по умолчанию `grsn`);
- имя роли – персональная роль с согласованными правами; владелец БД используется только для административных операций;
- пароль – секрет соответствующей роли, переданный по согласованному каналу.

В pgAdmin выберите регистрацию нового сервера, задайте удобное имя соединения и заполните вкладку подключения. В DBeaver создайте соединение PostgreSQL и укажите те же поля. Выполните проверку подключения, затем запрос:

```
SELECT current_user, current_database();
```

Сверьте роль и базу с заявкой. Пароль веб-пользователя АС ГРСН здесь не подходит. Для принудительной проверки парольного TCP-входа владельца из контейнера можно использовать команду ниже; `-W` запросит пароль интерактивно:

```
docker compose exec db psql -h 127.0.0.1 -U grsn -d grsn -W
```

Во всех примерах `grsn` – стандартные имя роли владельца и имя БД. Если они изменены в `.env`, подставьте свои значения. Номер порта 5432 внутри контейнера не меняется при изменении внешнего `DB_PORT`.

10.3 Удалённое подключение через SSH-туннель

SSH-сервер на хосте, персональная запись ОС и разрешение перенаправления портов должны быть настроены организацией заранее. АС ГРСН не устанавливает SSH. На рабочем месте администратора откройте терминал с клиентом OpenSSH и выполните:

```
ssh -N -L 127.0.0.1:15432:127.0.0.1:5432 sysadmin@grsn.example.org
```

Здесь `sysadmin` – ваша персональная запись ОС, `grsn.example.org` – сервер, 5432 – `DB_PORT` на сервере, а 15432 – свободный порт рабочего места. Если SSH слушает другой порт, добавьте параметр `-p` с его номером. При первом подключении проверьте отпечаток ключа сервера по данным ответственного администратора.

Оставьте терминал открытым. В клиенте PostgreSQL укажите хост `127.0.0.1`, порт `15432`, имя БД и свою роль PostgreSQL. SSH проверяет право подключения к ОС; затем PostgreSQL отдельно проверяет роль и пароль БД. Альтернативно заполните вкладку SSH-туннеля в самом клиенте БД. После работы закройте подключение и туннель. Порт PostgreSQL на внешнем интерфейсе сервера открывать не требуется.

10.4 Персональная роль для чтения справочников

Ниже приведён пример выдачи ограниченного доступа специалисту для чтения четырёх справочников. Роль создаётся администратором вручную и не является встроенной ролью приложения. Для другого задания состав таблиц согласуется отдельно. Чтение справочников не даёт права просматривать профили, запреты, рецепты, журнал и хеши паролей пользователей.

Откройте административную сессию:

```
docker compose exec db psql -U grsn -d grsn
```

Внутри `psql` выполните команды. Имя `contractor_read` замените персональным именем, а дату – согласованным сроком с часовым поясом; приведённая дата служит только примером. Пароль задаётся интерактивной командой `\password`:

```
CREATE ROLE contractor_read LOGIN
  NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION;
\password contractor_read
ALTER ROLE contractor_read
  VALID UNTIL '2026-12-01 18:00:00+08';
GRANT CONNECT ON DATABASE grsn TO contractor_read;
GRANT USAGE ON SCHEMA public TO contractor_read;
GRANT SELECT ON TABLE
  public.ingredients, public.ingredient_categories,
  public.allergens, public.fitness_goals
TO contractor_read;
```

Проверьте права из новой сессии этой роли, а не из сессии владельца:

```
SELECT current_user;
SELECT name FROM public.ingredients LIMIT 5;
SELECT has_table_privilege(
  current_user, 'public.ingredients', 'UPDATE');
SELECT has_table_privilege(
  current_user, 'public.users', 'SELECT');
```

Первый запрос должен показать персональную роль, второй – разрешённые строки, последние два – `false`. Не используйте для проверки разрушительные SQL-команды. Если роль раньше имела другие права или членства, проверьте и отзовите их отдельно. В этом примере права на будущие таблицы автоматически не выдаются.

Параметр `VALID UNTIL` ограничивает срок действия пароля при парольной аутентификации. Он не завершает уже открытые сессии и не заменяет отзыв доступа администратором в конце работ.

10.5 Отзыв прав и контроль изменений

После завершения задачи выполните в административной сессии БД:

```
ALTER ROLE contractor_read NOLOGIN;
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE usename = 'contractor_read'
  AND pid <> pg_backend_pid();
REVOKE SELECT ON TABLE
  public.ingredients, public.ingredient_categories,
  public.allergens, public.fitness_goals
FROM contractor_read;
REVOKE USAGE ON SCHEMA public FROM contractor_read;
REVOKE CONNECT ON DATABASE grsn FROM contractor_read;
```

`NOLOGIN` запрещает новые входы этой роли, а завершение соединений прекращает её текущие сеансы. Если выдавались другие разрешения или членства в ролях, отзовите их также. После проверки отсутствия зависимостей ненужную роль можно удалить; не удаляйте объекты данных ради удаления роли. Закройте SSH/VPN-доступ и выполните контрольное подключение с рабочего места исполнителя.

Изменение структуры таблиц выполняется миграциями Alembic при обновлении. Справочники редактируются через админку; прямые исправления SQL требуют отдельного согласования и копии БД. SQL-действия, выполненные через клиент PostgreSQL, не отражаются автоматически в журнале `audit_log` приложения.

База данных содержит 17 таблиц. Физическая модель базы данных представлена на рисунке 4.

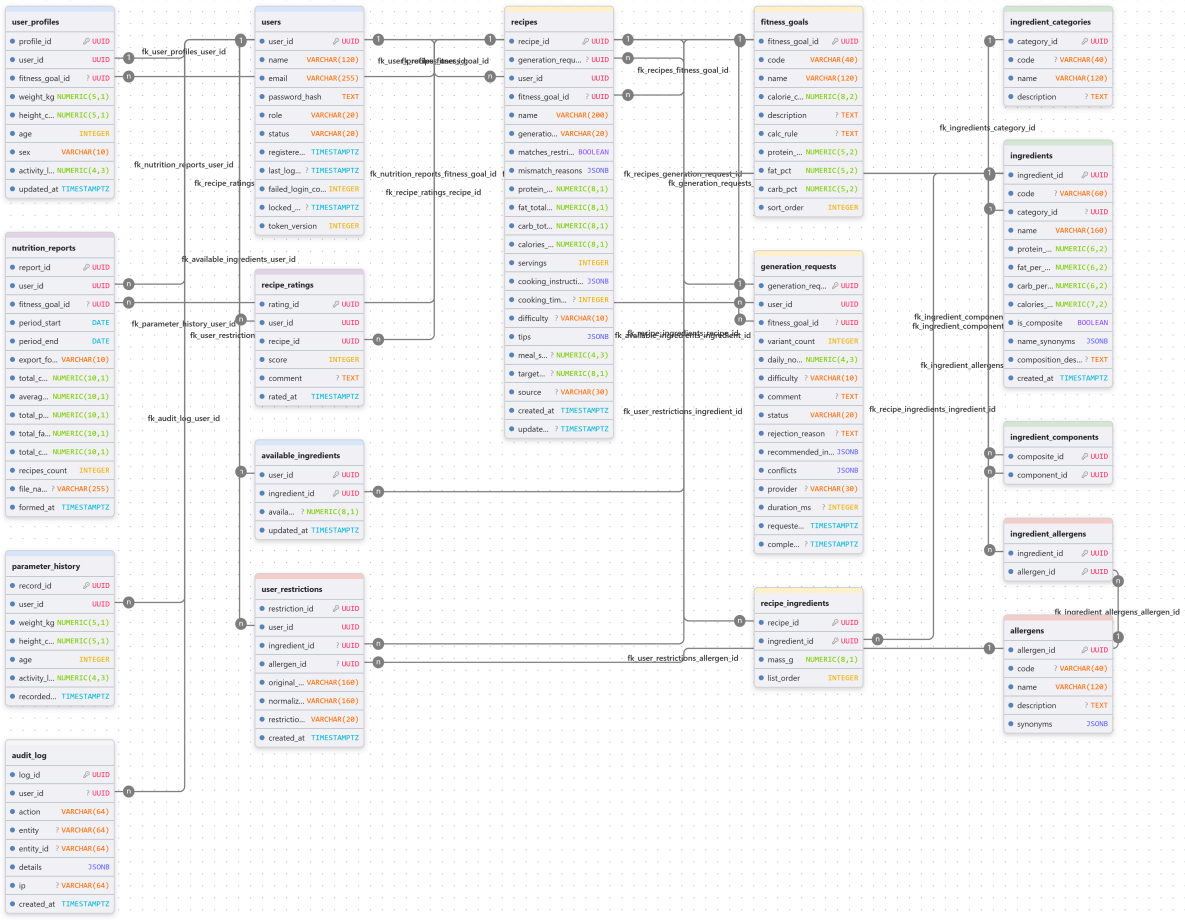


Рисунок 4 – Физическая модель базы данных

11 Модуль администрирования

Иллюстрации раздела показывают макеты интерфейса с условными значениями. Для выполнения операций используйте реальные записи своей системы; данные на рисунках не являются учётными данными для входа.

После установки войдите в систему под учётной записью администратора. В боковом меню появится раздел «Администрирование» (рисунок 5). В нём доступны вкладки:

– «Пользователи» – список учётных записей; изменение роли («Сделать админом»), блокировка и удаление пользователя, просмотр его активности;
– «Ингредиенты», «Категории», «Аллергены» – добавление, изменение и удаление записей справочников;
– «Логи» – журнал событий: входы и неудачные попытки входа, блокировки, действия пользователей и администратора, резервное копирование (рисунок 6);
– «Резервные копии» – список копий, создание копии и восстановление (раздел 9);
– «Целостность» – проверка корректности справочников: ингредиенты без категории, сумма белков, жиров и углеводов больше 100 г на 100 г продукта, расхождение калорийности с расчётом по белкам, жирам и углеводам, ошибки в составе составных продуктов, категории без ингредиентов, аллергены, не привязанные к ингредиентам, запреты пользователей без ссылки на справочник (рисунок 7).

11.1 Управление учётными записями

1) Откройте вкладку «Пользователи». В строке поиска введите имя или e-mail; сверьте найденную запись с заявкой.
2) Нажмите «Открыть», чтобы посмотреть последние события выбранного пользователя. Таблица также показывает роль, статус, последний вход и число рецептов.
3) Для выдачи полномочий нажмите «Сделать админом»; для их снятия – «Снять админа». Изменение применяется немедленно и завершает прежние сессии пользователя.
4) Для прекращения доступа нажмите «Блокировать». Для возврата доступа после согласования – «Разблокировать». Постоянная блокировка администратором и временная блокировка после подбора пароля – разные состояния; при временной блокировке дождитесь окончания указанного срока.
5) Удаляйте запись только после согласования удаления связанных данных: нажмите «Удалить» и подтвердите действие. Для временного прекращения доступа используйте блокировку.
6) Откройте вкладку «Логи» и проверьте, что изменение роли, статуса или удаление записано от вашей учётной записи.

Для новых сотрудников используется обычная регистрация, после которой назначаются права. Кнопки создания пользователя и задания ему нового пароля в административном интерфейсе нет.

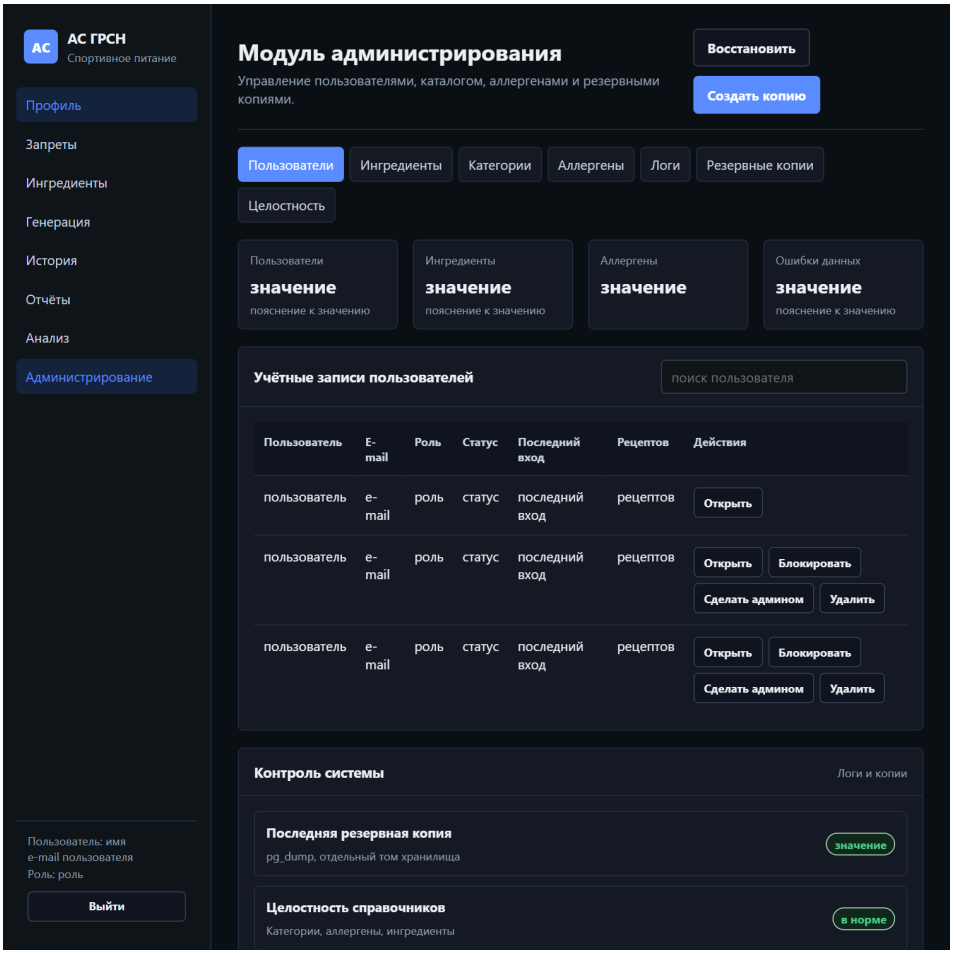


Рисунок 5 – Раздел «Администрирование», вкладка «Пользователи»

11.2 Наполнение каталога и справочников

Для добавления ингредиента:

1) Откройте вкладку «Ингредиенты» и нажмите «Добавить». В блоке «Новый ингредиент» заполните название и категорию.
2) Укажите белки, жиры и углеводы на 100 г продукта и калорийность на 100 г. Макронутриенты должны быть неотрицательными, каждый не более 100 г, сумма – не более 100 г; калорийность – от 0 до 950 ккал.
3) При необходимости заполните уникальный код НСИ и синонимы через запятую. Они используются для поиска и сопоставления названий запретов.
4) Отметьте связанные аллергены. Для составного продукта выберите компоненты из каталога и укажите описание состава; сам продукт нельзя включать в собственный состав, циклические ссылки недопустимы.
5) Нажмите «Сохранить ингредиент», проверьте уведомление и появление записи в каталоге, затем выполните проверку целостности.

Для исправления найдите строку, нажмите «Изменить», внесите значения и сохраните. «Отмена» закрывает редактирование. Удаление ингредиента, используемого в сохранённых рецептах, отклоняется: система сообщает причину.

Во вкладках «Категории» и «Аллергены» новая запись создаётся формой справа: наименование, код, описание; для аллергена также доступны синонимы. После «Сохранить» проверьте запись в таблице. Для изменения выберите «Изменить», для удаления – «Удалить» с подтверждением.

Удаление категории оставляет её ингредиенты без категории: назначьте им новую категорию и повторите проверку. Удаление аллергена, указанного в запретах пользователей, блокируется. Перед изменением связи продукта с аллергеном проверьте основание: эта связь влияет на проверку рецептов, а не только на отображение каталога.

11.3 Особенность штатного начального наполнения

При каждом запуске `backend` выполняется `app.seed`. Процедура находит штатные цели, категории, аллергены и ингредиенты по известным кодам и повторно присваивает им значения из поставки. Поэтому изменение штатных записей через админку может быть перезаписано после перезапуска или обновления; удалённые штатные записи могут появиться заново.

Перед редактированием штатной НСИ сохраните резервную копию и согласуйте с сопровождающим разработчиком обновление исходного наполнения. До такого согласованного изменения не считайте ручную правку штатной записи постоянной. Для новых записей используйте собственные уникальные коды, не совпадающие с кодами поставки; после любого перезапуска проверяйте сохранность согласованных изменений.

Эта особенность также действует после восстановления копии с последующим стартом API. В эксплуатационном журнале фиксируйте, какие изменения НСИ должны быть сохранены, и проверяйте их при приёмке обновления.

11.4 Просмотр журнала событий

Откройте вкладку «Логи», выберите тип событий и нажмите «Обновить» (рисунок 6). Доступны группы входов и регистрации, неудачных входов, генерации, действий администратора и резервного копирования. Интерфейс запрашивает до 300 последних записей выбранной группы; отсутствие более старой записи в таблице не доказывает отсутствие события.

Проверяйте время, пользователя, событие, детали и IP-адрес. Для копирования и восстановления различайте запуск, успешный результат и ошибку. Зафиксируйте сведения об инциденте в журнале эксплуатации. Действия напрямую в ОС, PostgreSQL и внешнем сервисе проверяются по их собственным журналам и отчётам работ.

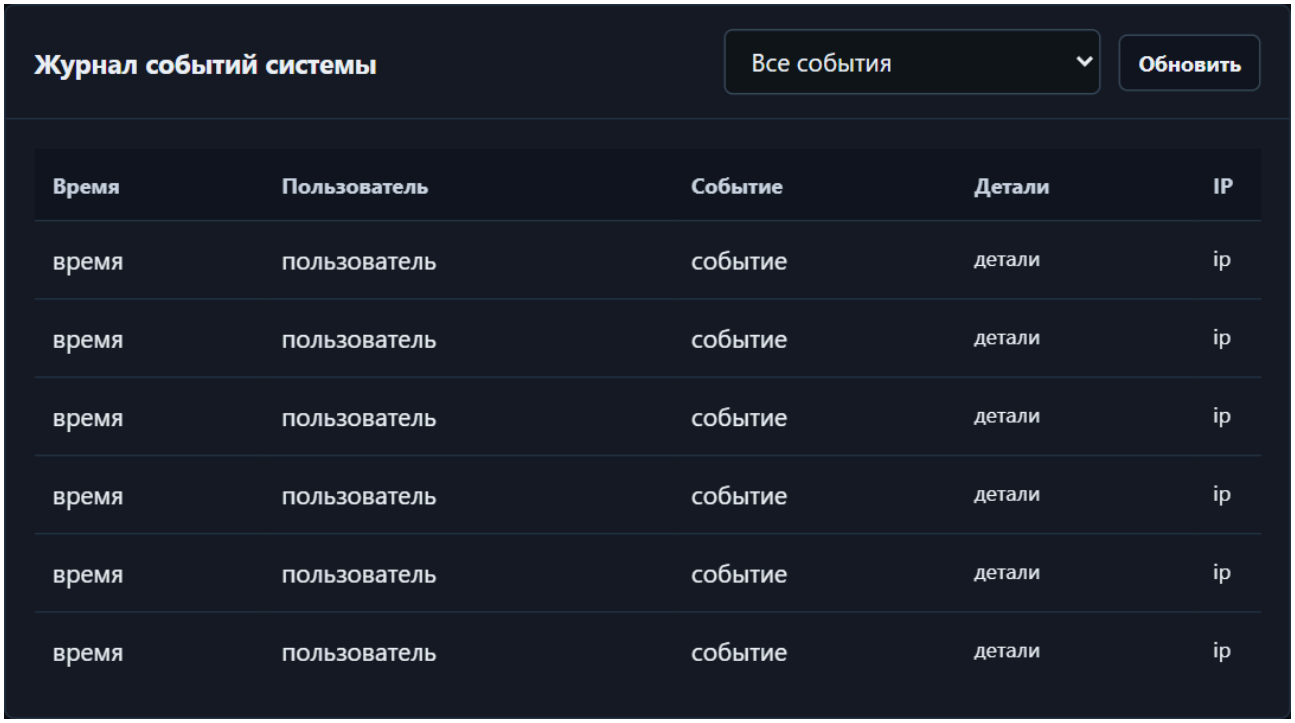


Рисунок 6 – Вкладка «Логи»

11.5 Проверка целостности и исправление замечаний

Откройте вкладку «Целостность» и нажмите «Проверить снова» (рисунок 7). Система покажет ошибки и предупреждения со ссылкой на тип данных. Исправления автоматически не выполняются.

– Для продукта без категории выберите категорию в редакторе ингредиента.
– Для суммы БЖУ больше 100 г сверьте исходные сведения и единицы измерения, затем исправьте запись.
– При расхождении калорийности с формулой Атуотера более чем на 20 % проверьте источник справочных значений. Предупреждение само по себе не требует механически заменять справочную калорийность расчётной.
– Для составного продукта без состава выберите компоненты и сохраните запись.
– Пустую категорию заполните либо удалите после проверки необходимости; непривязанный аллерген свяжите с нужными ингредиентами.
– Ошибку запрета без ссылки на справочник передайте ответственному за БД: прямые SQL-изменения без копии и согласования не выполняются.

После исправления повторите проверку и убедитесь, что замечание исчезло. Проверка охватывает реализованные правила справочников; она не заменяет проверку доступа, резервного восстановления и всех ограничений БД.

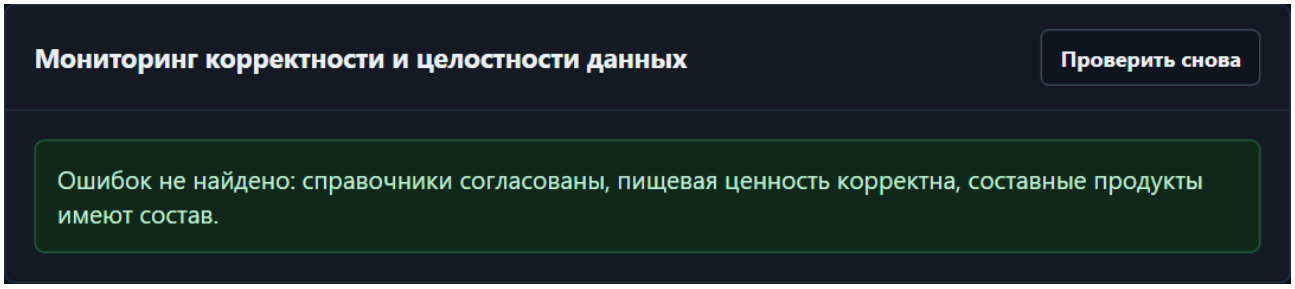


Рисунок 7 – Вкладка «Целостность»

Сразу после установки рекомендуется:

1) Войти под учётной записью администратора и проверить, что справочники заполнены, а вкладка «Целостность» не показывает ошибок.
2) Создать резервную копию вручную и убедиться, что файл появился в папке `BACKUP_PATH` на отдельном диске.
3) При необходимости назначить роль администратора ответственному сотруднику.

12 Обслуживание системы

Основные команды управления приведены в таблице 10. Команды выполняются в каталоге `grsn_app`.

Таблица 10 – Команды управления системой

Команда	**Назначение**
docker compose stop	Остановить систему (данные сохраняются)
docker compose up -d	Запустить систему или применить изменения .env
docker compose restart backend	Перезапустить отдельный контейнер
docker compose ps	Состояние контейнеров
docker compose logs -f backend	Журнал контейнера в реальном времени
docker stats	Потребление процессора и памяти
git pull и docker compose up -d --build	Обновление до новой версии: загрузка изменений и пересборка контейнеров; миграции базы применяются автоматически
docker compose down	Остановить и удалить контейнеры (данные в томах сохраняются)
docker compose down -v	Удалить контейнеры и именованные тома БД, отчётов и копий. Подключённая папка хоста не удаляется. Выполнять только после отдельного сохранения данных

Перед обновлением системы создайте резервную копию (раздел 9.2). Контейнеры настроены на автоматический перезапуск при сбое и после перезагрузки сервера (политика `restart: unless-stopped`), если служба Docker запускается вместе с системой.

Рекомендуемый регламент сопровождения:

– ежедневно – проверить состояние контейнеров, доступность сайта, наличие последней успешной копии и свободное место;
– еженедельно – проверить ошибки входа, административные действия, целостность справочников и завершённые доступы подрядчиков;
– ежемесячно – сверить действующие роли и сетевые разрешения с реестром, проверить сроки сертификатов и выполнить пробное восстановление;
– перед обновлением – согласовать окно, записать текущую версию кода, сохранить БД, файлы и конфигурацию, подготовить порядок возврата;
– после обновления или восстановления – выполнить проверки раздела 8, сверить НСИ и права пользователей. Копия может содержать прежние роли и статусы уже заблокированных сотрудников: повторно примените актуальные ограничения доступа. При необходимости принудительного завершения всех прежних сессий смените `JWT_SECRET`.

При подозрении на утечку доступа заблокируйте затронутые учётные записи на всех уровнях, завершите соединения, смените раскрытые секреты и сохраните журналы до устранения причины. Восстановление БД не заменяет отзыв скомпрометированных ключей ОС, VPN и внешних сервисов.

13 Типовые неисправности

Таблица 11 – Типовые неисправности и способы их устранения

Признак	**Причина и решение**
«Docker не найден» или «Docker не запущен»	Установите или запустите Docker (раздел 2.1) и повторите установку
docker compose up завершается ошибкой, порт занят	Укажите в .env свободные порты HTTP_PORT, HTTPS_PORT, DB_PORT либо освободите их
«Система не ответила за 5 минут»	Выполните docker compose logs backend; частые причины – нехватка памяти, неверный пароль базы данных после ручной правки .env
Сайт не открывается с других компьютеров	Откройте порты веб-интерфейса в межсетевом экране сервера
Браузер предупреждает о небезопасном соединении	Используется самоподписанный сертификат; установите сертификат, выданный для домена (раздел 3.3)
Пользователи видят предупреждение «рецепт сформирован локальным генератором»	Не задан или неверен ключ GigaChat, либо сервис недоступен: проверьте GIGACHAT_CREDENTIALS и доступ сервера к интернету, перезапустите backend
Генерация выполняется долго	Запросы к модели выполняются по очереди (тариф Freemium – один поток); при платном тарифе увеличьте LLM_MAX_CONCURRENCY
Резервная копия не создаётся, ошибка записи	Проверьте, что диск подключён и папка BACKUP_PATH доступна на запись пользователю 1001 (sudo chown 1001:1001 <папка>)
Заканчивается место на диске	Уменьшите BACKUP_RETENTION_DAYS, удалите старые образы командой docker image prune

Признак	Причина и решение
Администратор забыл пароль, других администраторов нет	Задайте в .env новые ADMIN_EMAIL и ADMIN_PASSWORD и выполните docker compose up -d backend: будет создана новая учётная запись администратора. Затем заблокируйте или удалите старую запись на вкладке «Пользователи»
API возвращает HTTP 401	Токен отсутствует, истёк или отозван. Выполните вход заново; проверьте время сервера. Не выдавайте права администратора для устранения ошибки входа
API возвращает HTTP 403	Проверьте статус записи и согласованную роль. Для группы admin нужна роль администратора; обычный пользователь не получает её автоматически
Клиент БД не подключается удалённо	Проверьте SSH/VPN, туннель, его локальный порт, имя роли и пароль БД. Прямая публикация PostgreSQL на внешний адрес для этого не требуется
Вход в БД успешен, SQL запрещён	Проверьте current_user и выданные права на нужные таблицы. Ограниченная персональная роль должна отклонять несогласованные операции
Доступ подрядчика остался после срока	Заблокируйте запись приложения и роль БД, завершите соединения, закройте VPN/SSH и внешние права по разделу 6.4. Веб-доступ по сроку сам не отключается
После перезапуска пропали правки НСИ	Проверьте, относится ли запись к штатному наполнению app.seed. Восстановите согласованные значения и согласуйте изменение поставки по разделу 11.3

14 Контроль готовности и журнал доступов

14.1 Контроль перед передачей в эксплуатацию

Ответственный администратор выполняет проверки из таблицы 12 и записывает результат. Это перечень действий при приёмке, а не утверждение о том, что они уже выполнены на конкретном сервере.

Таблица 12 – Контроль готовности системы к эксплуатации

№	Проверка	Ожидаемый результат
1	Запуск и /health	Четыре сервиса работают, БД и API готовы, статус ok
2	HTTPS с рабочего места	Домен и сертификат корректны, HTTP перенаправляется
3	Публикация портов	Веб доступен разрешённым сетям; API не опубликован отдельно, прямой доступ к БД извне закрыт
4	Первичный администратор и демо	Собственная запись ответственного; демонстрационная запись отключена или заблокирована
5	Вход и JWT	Корректный вход успешен; отсутствующий/отозванный токен не даёт доступа

№	Проверка	Ожидаемый результат
6	Разграничение ролей	Обычный пользователь не вызывает административные операции и не читает чужую историю
7	Персональный доступ к БД	Разрешённое чтение выполняется, несогласованные права отсутствуют
8	Отзыв пробного доступа	Блокировка прекращает веб-доступ; роль БД не входит, её сеансы завершены; туннель/внешние права закрыты
9	Интеграция GigaChat	Подтверждён фактический провайдер генерации либо зафиксирован согласованный локальный режим
10	Копия и восстановление	Копия на отдельном диске; пробное восстановление проверено в отдельной среде, файлы отчётов учтены
11	Журналы и НСИ	События доступны, согласованные справочники сохранены после перезапуска
12	Ответственные и секреты	Назначены владельцы доступов, известны порядок смены секретов и место их защищённого хранения

Проверки отказа во входе выполняйте на специально созданной тестовой записи, чтобы не заблокировать действующего администратора. Если проверка не пройдена, укажите причину, ответственного за исправление и срок повторной проверки.

14.2 Учёт выдачи и отзыва прав

Ведите реестр доступов в принятой организацией системе заявок или отдельном защищённом журнале. Для каждой записи сохраняйте:

– номер заявки, исполнителя и организацию, назначение доступа;
– согласовавшего и выдавшего права, дату и время начала и окончания с часовым поясом;
– уровень доступа: приложение, ОС, БД, VPN/SSH, репозиторий или внешний сервис;
– персональную учётную запись, роль и перечень разрешённых операций;
– разрешённый адрес/сеть и применённый способ аутентификации;
– даты фактического отзыва по каждому уровню, завершение активных сессий и результат контрольной проверки;
– ссылку на отчёт работ и записи журналов, если они требуются для задачи.

Пароли, JWT и закрытые ключи в реестр не включаются. Если срок доступа продлевается, оформляется новое согласование; отсутствие запроса на отзыв не означает автоматического продления. Раз в месяц сверяйте реестр с фактическими ролями приложения, PostgreSQL и внешних систем.

15 Источники технических сведений

Описание реализованных функций составлено по файлам поставки АС ГРСН: `docker-compose.yml`, `.env.example`, скриптам установки и копирования, модулям `config.py`, `security.py`, `deps.py`, `db_roles.py`, `seed.py`, маршрутам `auth` и `admin`, сервисам копирования и GigaChat, странице администрирования и клиенту сессии. При изменении этих компонентов инструкция подлежит актуализации.

Для настройки инфраструктуры и выполнения административных команд используются следующие первичные источники (дата обращения: 06.11.2026):

1) Docker Engine. Публикация и привязка портов – раздел 4.
2) Docker Engine. Фильтрация пакетов и межсетевые экраны – раздел 4.2.
3) PostgreSQL 15. Правила аутентификации pg_hba.conf – раздел 10.1.
4) PostgreSQL 15. CREATE ROLE и ALTER ROLE – создание и прекращение доступа, разделы 10.4–10.5.
5) PostgreSQL 15. GRANT – выдача прав на конкретные таблицы, раздел 10.4.
6) OpenSSH. Руководство ssh(1), параметр -L – защищённое перенаправление порта, раздел 10.3.
7) GigaChat API. Получение токена доступа – межсервисная авторизация, раздел 7.2.